

Cotizador (scaffold)

Proyecto scaffold para un cotizador con frontend en React (Vite) y backend en Node.js (Express).
Persistencia en filesystem (por defecto en `backend/outputs/`).

Novedades:

- Configuración de runtime del frontend vía `/config.js` (no requiere rebuild para cambiar URLs base)
- Página de diagnóstico protegida: `/admin/config` para ver la configuración efectiva
- Script de empaquetado cPanel: `npm run package:cpanel`
- LiveLog en tiempo real (solo admin) con respaldo NDJSON diario y WebSocket seguro (`/ws` y `/ws-public`). Ahora con:
 - Persistencia al refrescar (precarga los últimos ~50 eventos desde el NDJSON del día)
 - Filtros rápidos por nivel (info/warn/error), solo HTTP, métodos (GET/POST/PUT/PATCH/DELETE) y grupos de status (2xx/3xx/4xx/5xx)
 - Vista móvil más compacta (hora local, método, URL truncada y status)
- Toasts de alertas para usuarios (warn/error) vía canal público sanitizado
- Script para habilitar HTTPS local rápido: `npm run enable:https:dev`
- Archivo de ejemplo de entorno: `.env.example` (copiar a `.env`).
- Pipeline CI (GitHub Actions) que ejecuta lint, tests y build en pushes a `develop` y `main`.
- Inicio de sesión inmediato sin recarga: el guard de rutas espera hidratación y rehidrata desde `localStorage` si hay token, evitando rebotes y pantallas bloqueadas.
- Procesamiento asíncrono de PDF/email: crear/actualizar/copiar/aprobar/rechazar responden de inmediato; la generación de PDF y envío de email corren en background. Se emite `pdfReady` por SSE y la UI muestra "Generando PDF...".
- `/api/rates` resiliente: proxy del backend con caché local y degradación a 200 en caso de fallo (valores 0 y `source: 'unavailable'`). Control por `.env`.
- SSE más robusto: keep-alives y headers anti-buffering para operar detrás de proxies.

Guías útiles:

- Flujo de ramas: ver `BRANCH_FLOW_QUICKSTART.md` para una guía rápida de trabajo con main/develop/feature/hotfix).
- Manual de uso por roles: ver `USAGE.md` (Editor, Desarrollador y Mantenedor).

Instalación y ejecución (backend)

1. Instala dependencias en la raíz

```
npm install
```

2. Instala dependencias del frontend y arranca en modo desarrollo

```
cd frontend  
npm install  
npm run dev
```

3. Arranca el backend (desde la raíz)

```
npm run backend
```

4. Desarrollo todo-en-uno (frontend + backend con recarga y morgan)

```
npm run dev
```

5. Variables de entorno (crear `.env` basado en `.env.example`)

```
PORT=5000  
- Autenticación básica por email/contraseña (`/api/auth/register`,  
`/api/auth/login`, `/api/auth/me`) con roles `admin`, `cotizador`, `cliente`  
(enforcement opcional con `AUTHZ_STRICT=true`).  
- WhatsApp opcional vía API de Meta o Twilio: endpoint POST  
`/api/whatsapp/send`.  
- Envío de correo a múltiples destinatarios en `clientEmail` usando comas,  
punto y coma o espacios.  
SMTP_HOST=  
SMTP_PORT=  
SMTP_USER=  
SMTP_PASS=  
  
# Seguridad / Roles (opcional)  
JWT_SECRET=cambia-esto-por-una-clave-larga  
# Habilita enforcement de roles en middleware (por defecto false para no romper  
flujos)  
AUTHZ_STRICT=false  
  
# WhatsApp (opcional)  
# WHATSAPP_PROVIDER=meta|twilio
```

```

# META_WABA_TOKEN=...
# META_PHONE_NUMBER_ID=...
# TWILIO_ACCOUNT_SID=...
# TWILIO_AUTH_TOKEN=...
# TWILIO_WHATSAPP_FROM=whatsapp:+14155238886
MORGAN_FORMAT=dev
- Roles/Auth: el backend incluye rutas `/api/auth/*` (registro/login/me) y
middleware de roles. Por defecto `AUTHZ_STRICT=false` (no bloquea rutas).
Actívalo y aplica `requireRole` donde corresponda.
# Si el backend expone la API en otra URL/origen, define PUBLIC_API_BASE.
- `npm run deploy:vps`: despliegue monolítico al VPS con `deploy.vps.json`
(backend focalizado; útil cuando el frontend vive en cPanel).
PUBLIC_API_BASE=

# LiveLog y WebSockets
# Retención de logs diarios en días (por defecto 7)
LIVELOG_RETENTION_DAYS=7
# Alias soportado por compatibilidad
LIVELLOG_RETENTION_DAYS=7

# Rates (tipos de cambio) y timeouts
# Fuente preferida para el frontend (vía backend)
RATES_SOURCE=backend
# Saltar consulta de tasas durante creación/actualización de cotizaciones (útil
en contingencia)
QUOTE_SKIP_RATES=0
PDF_SKIP_RATES=0
# Timeout de la consulta a la fuente (ms)
RATES_TIMEOUT_MS=3000

```

- Roles opcional: **AUTHZ_STRICT** (enforcement de roles admin/cotizador/cliente). Rutas de auth: `/api/auth/register|login|me`.
- WhatsApp (opcional): **WHATSAPP_PROVIDER=meta|twilio** y credenciales del proveedor.

7. Lint antes de construir

- Multi-destinatario: en el campo "Email Cliente" del editor, puedes ingresar varios correos separados por coma, punto y coma o espacios. El backend los validará y enviará a todos los válidos.

```
npm run lint ; npm run frontend:build
```

Puedes integrar un script combinado si lo prefieres:

```

{
  "scripts": {
    "build:checked": "npm run lint && npm run frontend:build"
  }
}

```

8. Pruebas locales y en la nube

Dispones de tres suites de pruebas:

- Backend local (unit/smoke):

```
npm run test:backend
```

- Frontend (Vitest):

```
npm run test:frontend
```

- Seguridad HTTPS local (recorre rutas y verifica mixed content):

```
npm run test:security
```

- Seguridad E2E en producción (frontend y backend públicos):

```
# opcional: overrides
$env:FRONTEND_URL_PROD="https://cotizador.aysafi.com";
$env:BACKEND_URL_PROD="https://emqx.aysafi.com:8443"; npm run
test:security:prod
```

Notas e interpretación:

- Si el backend 8443 no está alcanzable, las pruebas de backend se marcarán como SKIP y no fallarán la suite; revisa Nginx/firewall/servicio.
- Si `/admin/login` devuelve 404 en el frontend productivo, es indicio de que falta el fallback SPA del servidor estático; corrige `.htaccess`.
- Si no existe `/config.js` en el frontend, es válido (config embebida). El test sólo lanza advertencia.
- Si hay advertencia de referencias `http://` en assets, revisa el bundle o CSP. En producción se recomienda CSP para mitigar inyecciones.

HTTPS local (rápido) y WS seguros

Para probar WSS y evitar mixed content:

```
npm run enable:https:dev
$env:HTTPS="true"; $env:TLS_CERT_FILE=".\\backend\\certs-dev\\localhost.crt";
$env:TLS_KEY_FILE=".\\backend\\certs-dev\\localhost.key";
```

```
$env:TLS_CA_FILE=".\\backend\\certs-dev\\localhost.ca-bundle.crt"; npm run backend
```

El frontend detecta `https:` y usa `wss:` para `/ws` y `/ws-public`.

LiveLog (admin) y Alertas (usuarios)

- Vista: `/admin/livelog` (protegida). Requiere `admin_token` (login en `/admin/login`).
- Canal admin: `wss://<host>/ws?token=<JWT>` → todos los eventos (request, console, etc.).
- Canal público: `wss://<host>/ws-public` → solo warn/error con `msg` y `type` sanitizados.
- Archivos NDJSON rotados por día en `backend/outputs/logs/`. Retención: `LIVELOG_RETENTION_DAYS` (default 7).

CI / Verificación rápida local

Para ejecutar la misma verificación que el pipeline CI (sin pruebas de producción) puedes usar:

```
npm run ci:verify
```

Incluye: lint, tests backend, tests frontend y build del frontend.